

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 7

дисциплина: Архитектура компьютера

Студент: Савенкова Татьяна Александровна

Группа: НКАбд-05-25

МОСКВА

2025 г.

Оглавление

1 Цель работы	4
2 Задание	5
3 Теоретическое введение	6
4 Выполнение лабораторной работы.....	7
4.1 Реализация переходов в NASM.....	7
4.2 Изучение структуры файла листинга	11
4.3 Задания для самостоятельной работы	14
5 Выводы	17
Список литературы	18

Список иллюстраций

Рисунок 1 Создание каталога и файла для программы	7
Рисунок 2 Сохранение программы	7
Рисунок 3 Запуск программы.....	8
Рисунок 4 Изменение программы.....	8
Рисунок 5 Запуск измененной программы	9
Рисунок 6 Изменение программы.....	9
Рисунок 7 Проверка изменений	10
Рисунок 8 Сохранение новой программы	10
Рисунок 9 Проверка программы из листинга	11
Рисунок 10 Проверка файла листинга.....	12
Рисунок 11 Удаление операнда из программы	13
Рисунок 12 Просмотр ошибки в файле листинга.....	13
Рисунок 13 Первая программа самостоятельной работы.....	14
Рисунок 14 Проверка работы первой программы	15
Рисунок 15 Вторая программа самостоятельной работы	15
Рисунок 16 Проверка работы второй программы	16

1 Цель работы

Изучение команд условного и безусловного переходов. Приобретение навыков написания программ с использованием переходов. Знакомство с назначением и структурой файла листинга.

2 Задание

1. Реализация переходов в NASM
2. Изучение структуры файлов листинга
3. Самостоятельное написание программ по материалам лабораторной работы.

3 Теоретическое введение

Для реализации ветвлений в ассемблере используются так называемые команды передачи управления или команды перехода. Можно выделить 2 типа переходов:

- условный переход – выполнение или не выполнение перехода в определенную точку программы в зависимости от проверки условия.
- безусловный переход – выполнение передачи управления в определенную точку программы без каких-либо условий.

4 Выполнение лабораторной работы

4.1 Реализация переходов в NASM

Создаю каталог для программ лабораторной работы №7 (Рисунок 1).

```
tasavenkova@fedora:~$ mkdir ~/work/arch-pc/lab07
tasavenkova@fedora:~$ cd ~/work/arch-pc/lab07
tasavenkova@fedora:~/work/arch-pc/lab07$ touch lab7-1.asm
tasavenkova@fedora:~/work/arch-pc/lab07$ mc
```

Рисунок 1 Создание каталога и файла для программы
Копирую код из листинга в файл будущей программы. (Рисунок 2).

```
%include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label2
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'
_end:
call quit
```

Рисунок 2 Сохранение программы

При запуске программы я убедился в том, что безусловный переход действительно изменяет порядок выполнения инструкций (Рисунок 3).

```
tasavenkova@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
tasavenkova@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
tasavenkova@fedora:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 3
tasavenkova@fedora:~/work/arch-pc/lab07$
```

Рисунок 3 Запуск программы

Изменяю программу таким образом, чтобы поменялся порядок выполнения функций (Рисунок 4).

```
%include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label2
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
jmp _end
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
jmp _label1
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'
_end:
call quit
```

Рисунок 4 Изменение программы

Запускаю программу и проверяю, что примененные изменения верны (Рисунок 5).


```

tasavenkova@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
tasavenkova@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
tasavenkova@fedora:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 1
tasavenkova@fedora:~/work/arch-pc/lab07$ 

```

Рисунок 5 Запуск измененной программы

Теперь изменяю текст программы так, чтобы все три сообщения вывелись в обратном порядке (Рисунок 6).

```

%include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label2
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
jmp _end
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
jmp _label1
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'
_end:
call quit

```

Рисунок 6 Изменение программы

Работа выполнена корректно, программа в нужном мне порядке выводит сообщения (Рисунок 7).

```

tasavenkova@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
tasavenkova@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
tasavenkova@fedora:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 3
Сообщение № 2
Сообщение № 1
tasavenkova@fedora:~/work/arch-pc/lab07$

```

Рисунок 7 Проверка изменений

Создаю новый рабочий файл и вставляю в него код из следующего листинга

(Рисунок 8).

```

#include 'in_out.asm'
section .data
msg1 db 'Введите B: ',0h
msg2 db "Наибольшее число: ",0h
A dd '20'
C dd '50'
section .bss
max resb 10
B resb 10
section .text
global _start
_start:
; ----- Вывод сообщения 'Введите B: '
mov eax,msg1
call sprint
; ----- Ввод 'B'
mov ecx,B
mov edx,10
call sread
; ----- Преобразование 'B' из символа в число
mov eax,B
call atoi ; Вызов подпрограммы перевода символа в число
mov [B],eax ; запись преобразованного числа в 'B'
; ----- Записываем 'A' в переменную 'max'
mov ecx,[A] ; 'ecx = A'
mov [max],ecx ; 'max = A'
; ----- Сравниваем 'A' и 'C' (как символы)
cmp ecx,[C] ; Сравниваем 'A' и 'C'
jg check_B ; если 'A>C', то переход на метку 'check_B',
mov ecx,[C] ; иначе 'ecx = C'
mov [max],ecx ; 'max = C'
; ----- Преобразование 'max(A,C)' из символа в число
check_B:
mov eax,max
call atoi ; Вызов подпрограммы перевода символа в число
mov [max],eax ; запись преобразованного числа в 'max'
; ----- Сравниваем 'max(A,C)' и 'B' (как числа)

```

Рисунок 8 Сохранение новой программы

Программа выводит значение переменной с максимальным значением, проверяю работу программы с разными входными данными (Рисунок 9).

```
tasavenkova@fedora:~/work/arch-pc/lab07$ ./lab7-2
Введите В: 5
Наибольшее число: 50
tasavenkova@fedora:~/work/arch-pc/lab07$ ./lab7-2
Введите В: 100
Наибольшее число: 100
tasavenkova@fedora:~/work/arch-pc/lab07$ ./lab7-2
Введите В: 78
Наибольшее число: 78
tasavenkova@fedora:~/work/arch-pc/lab07$
```

Рисунок 9 Проверка программы из листинга

4.2 Изучение структуры файла листинга

Создаю файл листинга с помощью флага -l команды nasm и открываю его с помощью текстового редактора mcedit (Рисунок 10).

```

lab7-2.lst      [----]  0 L: [ 1+ 0  1/225] *(0  /14458b) 0032 0x020      [*][X]
1               %include 'in_out.asm'
1               <1> ;----- slen -----
2               <1> ; Функция вычисления длины сообщения
3               <1> slen:.....
4 00000000 53    <1>      push    ebx.....
5 00000001 89C3  <1>      mov     ebx, eax.....
6               <1>.....
7               <1> nextchar:.....
8 00000003 803800 <1>      cmp     byte [eax], 0...
9 00000006 7403   <1>      jz      finished.....
10 00000008 40    <1>      inc     eax.....
11 00000009 EBF8  <1>      jmp     nextchar.....
12               <1>.....
13               <1> finished:
14 0000000B 29D8  <1>      sub     eax, ebx
15 0000000D 5B    <1>      pop     ebx.....
16 0000000E C3    <1>      ret.....
17               <1>.....
18               <1>.....
19               <1> ;----- sprint -----
20               <1> ; Функция печати сообщения
21               <1> ; входные данные: mov eax,<message>
22               <1> sprint:
23 0000000F 52    <1>      push    edx
24 00000010 51    <1>      push    ecx
25 00000011 53    <1>      push    ebx
26 00000012 50    <1>      push    eax
27 00000013 E8E8FFFFFF <1>      call    slen
28               <1>.....
29 00000018 89C2  <1>      mov     edx, eax
30 0000001A 58    <1>      pop     eax
31               <1>.....
32 0000001B 89C1  <1>      mov     ecx, eax
33 0000001D B801000000 <1>      mov     ebx, 1
34 00000022 B804000000 <1>      mov     eax, 4
35 00000027 CD80  <1>      int     80h
36               <1>.....
37 00000029 5B    <1>      pop     ebx

```

1Помощь 2Сохран 3Блок 4Замена 5Копия 6Пере-ить 7Поиск 8Удалить 9МенюМС 10Выход

Рисунок 10 Проверка файла листинга

Первое значение в файле листинга - номер строки, и он может вовсе не совпадать с номером строки изначального файла. Второе вхождение - адрес, смещение машинного кода относительно начала текущего сегмента, затем непосредственно идет сам машинный код, а заключает строку исходный текст программы с комментариями.

Удаляю один операнд из случайной инструкции, чтобы проверить поведение файла листинга в дальнейшем (Рисунок 11).

```

GNU nano 8.3 /home/tasavenkova/work/arch-pc/lab07/lab7-2.asm
#include 'in_out.asm'
section .data
msg1 db 'Введите B: ',0h
msg2 db "Наибольшее число: ",0h
A dd '20'
C dd '50'
section .bss
max resb 10
B resb 10
section .text
global _start
_start:
; ----- Вывод сообщения 'Введите B: '
mov eax,msg1
call sprint
; ----- Ввод 'B'
mov ecx,B
mov edx,10
call sread
; ----- Преобразование 'B' из символа в число
mov eax,
call atoi ; Вызов подпрограммы перевода символа в число
mov [B],eax ; запись преобразованного числа в 'B'
; ----- Записываем 'A' в переменную 'max'
mov ecx,[A] ; 'ecx = A'
mov [max],ecx ; 'max = A'
; ----- Сравниваем 'A' и 'C' (как символы)
cmp ecx,[C] ; Сравниваем 'A' и 'C'
jg check_B ; если 'A>C', то переход на метку 'check_B',
mov ecx,[C] ; иначе 'ecx = C'
mov [max],ecx ; 'max = C'
; ----- Преобразование 'max(A,C)' из символа в число
check_B:
mov eax,max
call atoi ; Вызов подпрограммы перевода символа в число
mov [max],eax ; запись преобразованного числа в 'max'

^G Справка      ^O Записать     ^F Поиск        ^K Вырезать     ^T Выполнить    ^C Позиция      M-U Отмена
^X Выход        ^R ЧитФайл     ^\ Замена       ^U Вставить     ^J Выровнять    ^/ К строке     M-E Повтор

```

Рисунок 11 Удаление операнда из программы

В новом файле листинга показывает ошибку, которая возникла при попытке трансляции файла. Никакие выходные файлы при этом помимо файла листинга не создаются. (Рисунок 12).

```

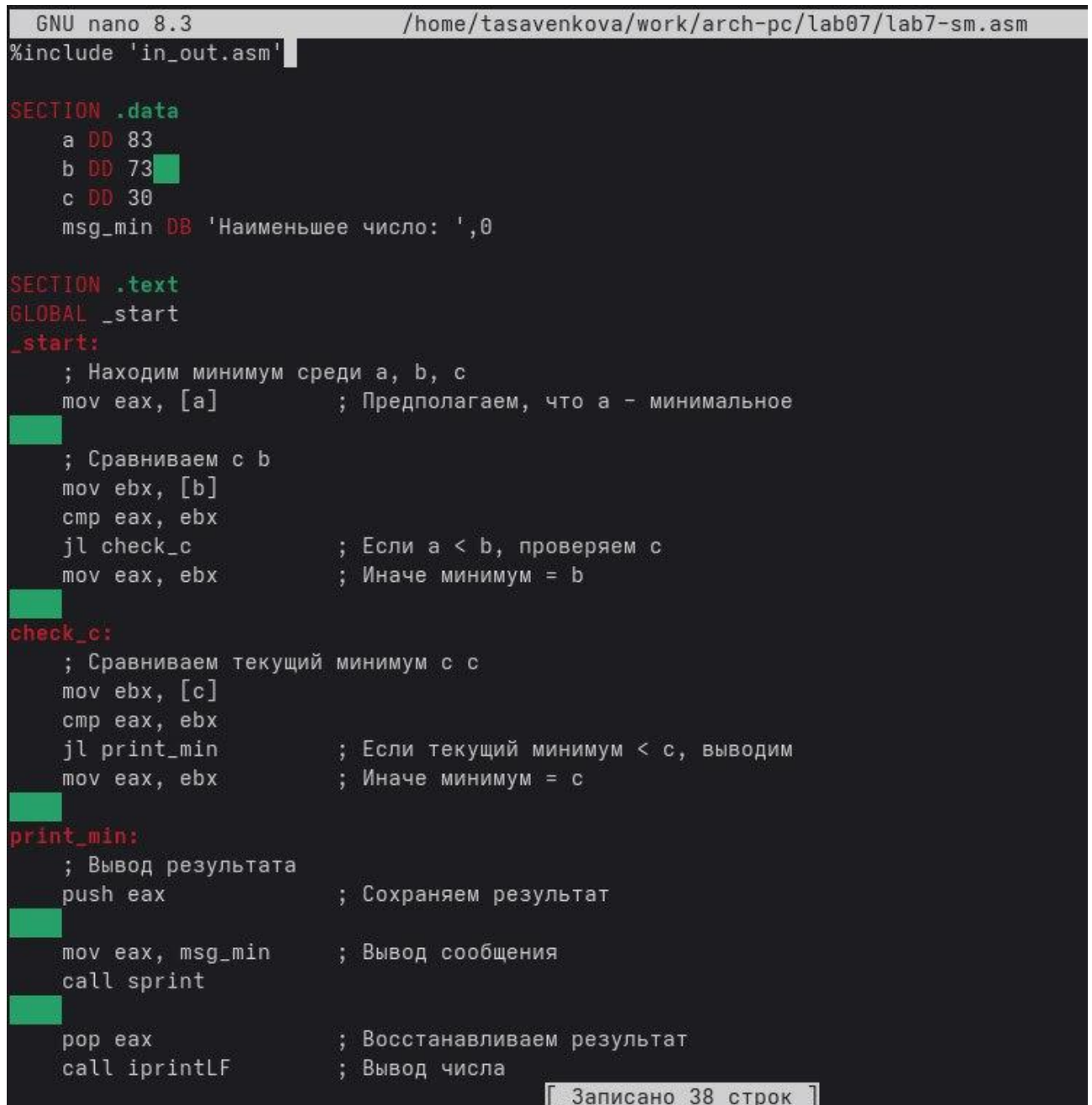
tasavenkova@fedora:~/work/arch-pc/lab07$ nasm -f elf -l lab7-2.asm
nasm: fatal: no input file specified
Type nasm -h for help.
tasavenkova@fedora:~/work/arch-pc/lab07$

```

Рисунок 12 Просмотр ошибки в файле листинга

4.3 Задания для самостоятельной работы

Буду выполнять вариант 18, та как в 6 лабораторной мне выдали 18 вариант. Возвращаю операнд к функции в программе и изменяю ее так, чтобы она выводила переменную с наименьшим значением (Рисунок 13).



```
GNU nano 8.3 /home/tasavenkova/work/arch-pc/lab07/lab7-sm.asm
#include 'in_out.asm'

SECTION .data
a DD 83
b DD 73
c DD 30
msg_min DB 'Наименьшее число: ',0

SECTION .text
GLOBAL _start
_start:
    ; Находим минимум среди a, b, c
    mov eax, [a]          ; Предполагаем, что a - минимальное

    ; Сравниваем с b
    mov ebx, [b]
    cmp eax, ebx
    jl check_c            ; Если a < b, проверяем c
    mov eax, ebx          ; Иначе минимум = b

check_c:
    ; Сравниваем текущий минимум с c
    mov ebx, [c]
    cmp eax, ebx
    jl print_min          ; Если текущий минимум < c, выводим
    mov eax, ebx          ; Иначе минимум = c

print_min:
    ; Вывод результата
    push eax              ; Сохраняем результат

    mov eax, msg_min      ; Вывод сообщения
    call sprint

    pop eax               ; Восстанавливаем результат
    call iprintLF         ; Вывод числа

[ Записано 38 строк ]
```

Рисунок 13 Первая программа самостоятельной работы

Проверяю корректность написания первой программы (Рисунок 14).

```

tasavenkova@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-sm.asm
tasavenkova@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-sm lab7-sm.o
tasavenkova@fedora:~/work/arch-pc/lab07$ ./lab7-sm
Наименьшее число: 30
tasavenkova@fedora:~/work/arch-pc/lab07$ █

```

Рисунок 14 Проверка работы первой программы

Пишу программу, которая будет вычислять значение заданной функции согласно моему варианту для введенных с клавиатурных переменных а и х (Рисунок 15).

```

GNU nano 8.3 /home/tasavenkova/work/arch-pc/lab07/lab7-sm2.asm
#include 'in_out.asm'

SECTION .data
msg_x DB 'Введите x: ', 0
msg_a DB 'Введите a: ', 0
msg_result DB 'Результат: ', 0
newline DB 10, 0

SECTION .bss
x RESD 1
a RESD 1
result RESD 1

SECTION .text
GLOBAL _start
_start:
    ; Ввод значения x
    mov eax, msg_x
    call sprint
    call sread
    call atoi
    mov [x], eax

    ; Ввод значения a
    mov eax, msg_a
    call sprint
    call sread
    call atoi
    mov [a], eax

    ; Проверка условия a = 1
    mov ebx, [a]
    cmp ebx, 1
    je case_a_equal_1    ; Если a = 1, переходим к соответствующей ветке

    ; Случай a ≠ 1: f(x) = a²

```

Рисунок 15 Вторая программа самостоятельной работы

Транслирую и компоную файл, запускаю и проверяю работу программы для различных значений а и х (Рисунок 16).

```
tasavenkova@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-sm2.asm
tasavenkova@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-sm2 lab7-sm2.o
tasavenkova@fedora:~/work/arch-pc/lab07$ ./lab7-sm2
Введите x: 1
Введите a: 2
f(x) = 4
tasavenkova@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-sm2.asm
tasavenkova@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-sm2 lab7-sm2.o
tasavenkova@fedora:~/work/arch-pc/lab07$ ./lab7-sm2
Введите x: 2
Введите a: 1
f(x) = 12
tasavenkova@fedora:~/work/arch-pc/lab07$
```

Рисунок 16 Проверка работы второй программы

5 Выводы

При выполнении лабораторной работы я изучил команды условных и безусловных переходов, а также приобрел навыки написания программ с использованием переходов, познакомился с назначением и структурой файлов листинга.

Список литературы

1.[Курс на ТУИС](#)

2.[Лабораторная работа №7](#)

3.[Программирование на языке ассемблера NASM Столяров А. В.](#)